Version 1.08
09/28/2026

NMC Bring-Up Guide: Cabling, Programming & Loading Linux

WARNING: Wear an ESD wrist strap whenever you handle the NMC board. Electrostatic discharge can permanently damage the board.

Overview

Going from a bare NMC board to a booting Linux prompt takes eight steps and three connections to your host PC: a programming connection for the FPGA design and firmware, USB-to-serial connections for the two serial consoles, and one Ethernet cable for TFTP.

1. Install Libero SoC, its license, and SoftConsole.
2. Clone the Nextage `Project-NMC` repo and learn which MSS output files go where.
3. Cable the board and open two serial terminals.
4. Program the FPGA design and firmware (Libero, MSS Configurator, SoftConsole).
5. Build the Linux images, or use the prebuilt ones.
6. Flash `payload.bin` to eMMC from the HSS prompt (YMODEM).
7. Flash `sdcard.img` to eMMC from U-Boot (TFTP).
8. Boot Linux and persist auto-boot.

Have on hand:

- The NMC board and a 5 V power supply, current-limited to 2 A
- A Windows host with WSL2 / Ubuntu 22.04 LTS (the tested setup for the Linux build)
- Tera Term, a TFTP server such as tftpd64, and GitHub Desktop
- A Microchip account for the Libero license download
- The cables listed in Step 3

Step 1: Install the software

Install Libero SoC (with its node-locked license) and SoftConsole on the host PC; both use default download settings.

Libero SoC Design Suite (includes the MSS Configurator)

1. Download Libero SoC from Microchip's FPGA Software Downloads page. Keep the default settings and pick any install directory.
2. Libero needs a license tied to your machine. Generate it from Microchip's FPGA Software Licenses page and follow the *Libero SoC License Installation Guide* for the **1 year node-locked Silver license**.
3. Node-locked means the license is bound to this one device. Floating licenses are for servers.
4. Once the license is installed and linked, Libero SoC should launch and run.

SoftConsole

1. Download SoftConsole from Microchip's SoftConsole page.
2. Keep the default settings and pick any install directory.

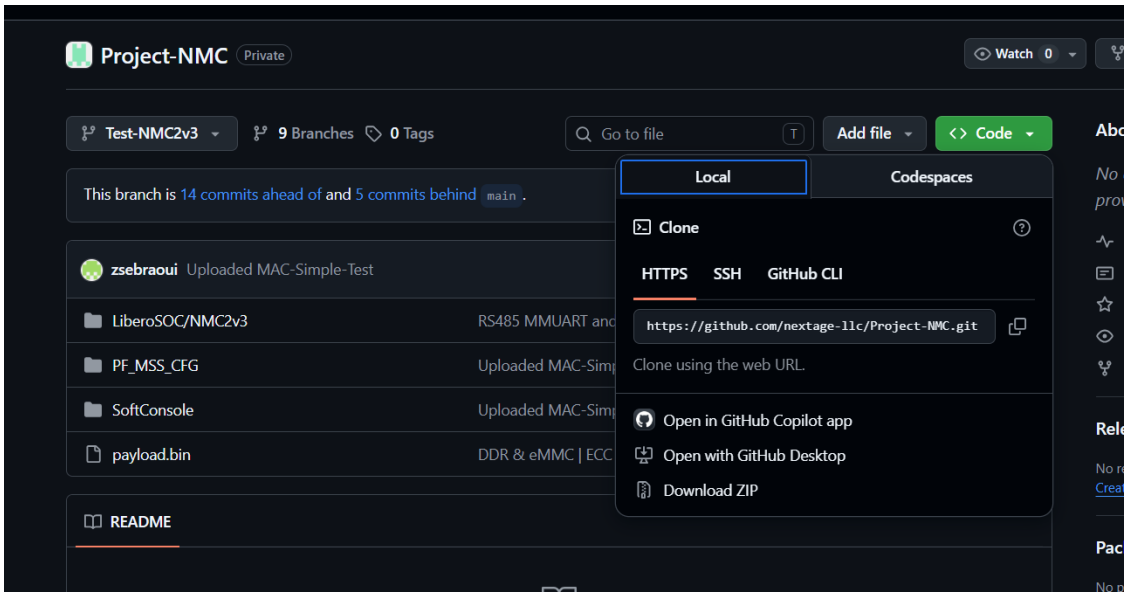
Links

- Libero SoC download (FPGA Software Downloads)
  https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/fpga-software-downloads
- Libero license generation (FPGA Software Licenses)
  https://www.microchipdirect.com/fpga-software-products
- Libero SoC License Installation Guide
  https://ww1.microchip.com/downloads/aemDocuments/documents/FPGA/swdocs/libero/Libero_Installation_Licensing_Setup_User_Guide_2025_1.pdf
- SoftConsole download
  https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/soc-fpga/softconsole

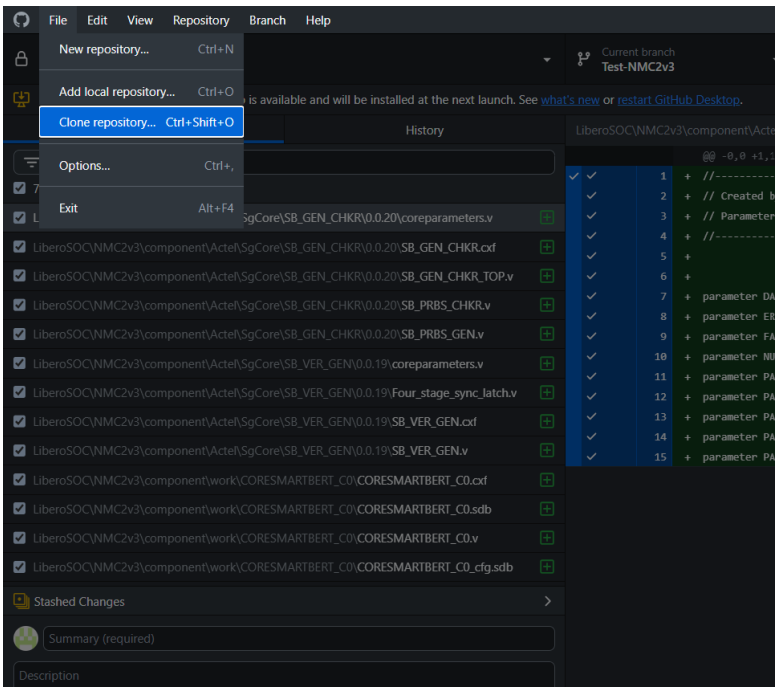
Step 2: Get the code and know your files

Clone one branch of the Nextage `Project-NMC` repo into an empty folder; Install GitHub Desktop.

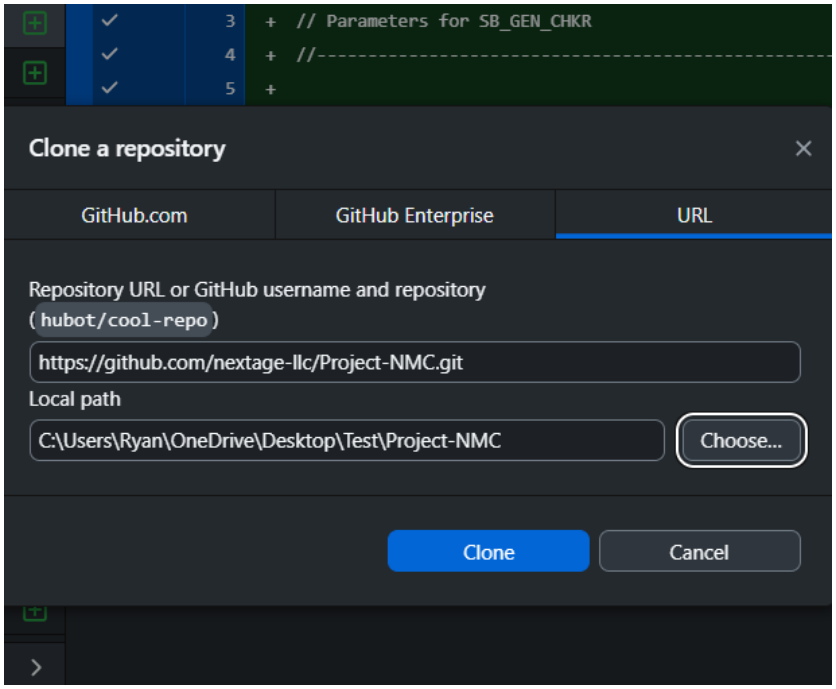
1. Choose **File → Clone repository → URL** and enter `https://github.com/nextage-llc/NMC_OS_BSP.git`.
2. Set the local path to a new, empty folder.
3. Pick the branch you need. `main` is the baseline and the others are test branches (the guide's screenshots use `Test-NMC2v3`). Click **Clone**.



On the repo page, the Code menu shows the clone URL and an "Open with GitHub Desktop" option. The branch selector at top left picks the branch (here Test-NMC2v3).



In GitHub Desktop, open File → Clone repository (Ctrl+Shift+O).



On the URL tab, paste the repo URL, set Local path to your empty folder, and click Clone.

Each branch contains `LiberoSOC/NMC2v3`, `PF_MSS_CFG`, `SoftConsole`, and a `payload.bin`.

The MSS Configurator produces files that both Libero and SoftConsole consume. The `.hex` comes from an IDE such as SoftConsole instead.

File	What it is	Where it goes
`.cfg`	GUI settings file for the RISC-V MSS. Not used in firmware generation.	Open it in the MSS Configurator to edit MSS settings. Press **Generate** to produce the `.cxz` and `.xml`.
`.cxz`	MSS Configurator output	Libero SoC design flow: **Create Design → Import MSS**
`.xml`	MSS Configurator output; carries the hardware configuration to the RISC-V processors	SoftConsole: drag it into the project and delete the default `.xml`, so only the new one remains
`.hex`	Built in an IDE (SoftConsole); used for bootloading and memory initialization	Libero SoC: **Program Design → Configure Design Initialization Data and Memories → eNVM**

The separate Linux BSP lives in its own repo, `mpfs-nmc-linux-bsp`, covered in Step 5.

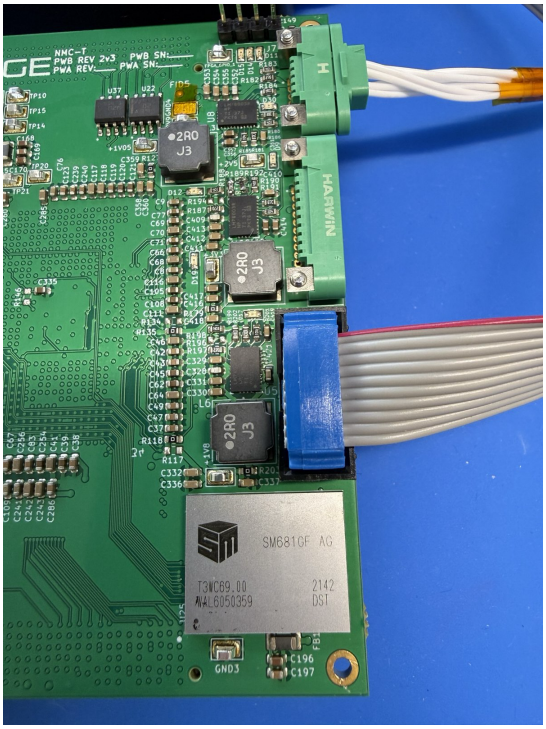
Step 3: Cable the board and open the serial terminals

Connect the board to the host PC with USB-to-serial cabling into the UART0 and UART1 headers for the consoles and one Ethernet cable for TFTP, plus a Microchip FlashPro programmer and a 5 V power feed (photos below).

Connection	From	To	Used for	Source detail
Power	5 V supply	Board (J7)	Powering and power-cycling the board	Photo below
Programming link	Host PC (Libero), then USB to FlashPro5 or FlashPro6	Board (J2)	Programming the FPGA design and firmware (Step 4)	Photos below
USB	Host PC (USB)	Board (UART0 and UART1 headers)	Two serial ports: UART0 (HSS bootloader) and UART1 (U-Boot and Linux console)	Photo below
Ethernet	Host PC	Board (J19)	TFTP transfer of `sdcard.img` (Step 7)	Photo below. Direct to the host.

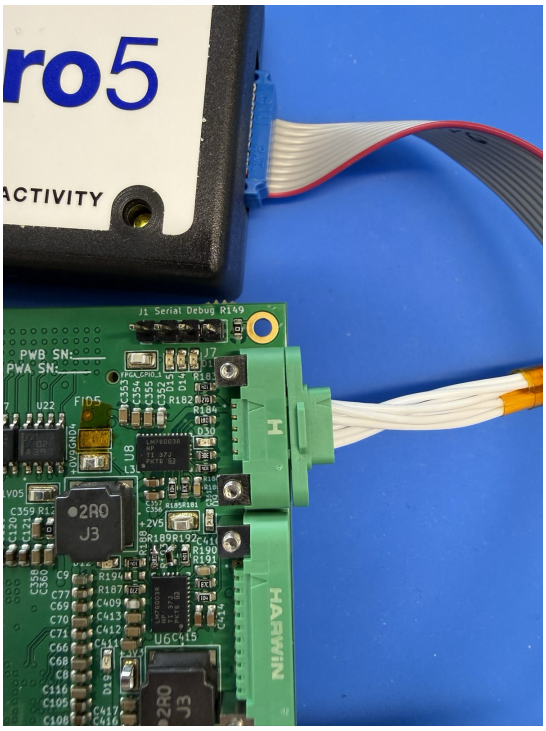
Connect the programmer. Plug the FlashPro5 or FlashPro6 into a USB port on the host PC, then plug its ribbon cable into the J2 connector on the NMC's edge.

The FlashPro5 (a FlashPro6 works the same way). The USB cable to the host PC leaves on the left, and the ribbon cable to the NMC leaves on the right.



The ribbon cable seats in the J2 connector on the board edge. Match the orientation in the photo, with the cable's red-stripe edge on top.

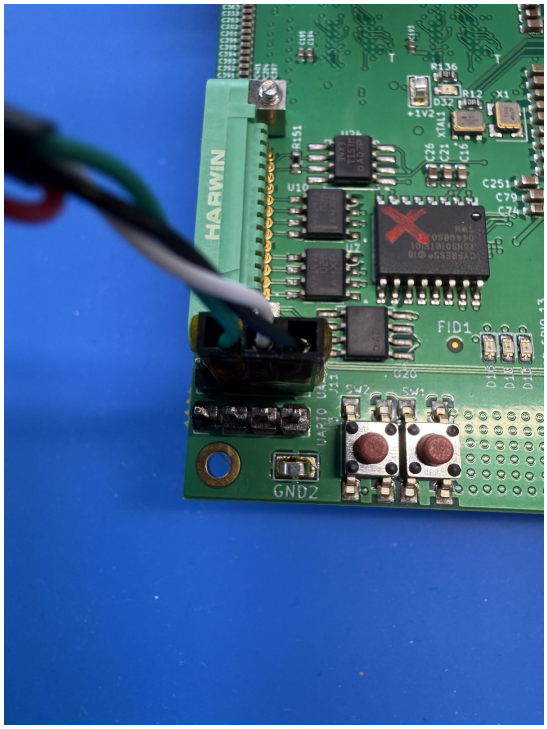
Connect power. Feed 5V into connector J7 on the board edge. Set the supply's current limit to 2 A. There are no jumpers or boot-mode settings to set and no power-up order: turn on the supply and the board sequences its own internal rails.



Power goes into J7, shown here with the white wire bundle plugged in.

Connect the serial consoles. Connect the USB-to-serial cable to the board's UART headers, UART0 for the HSS bootloader and UART1 for U-Boot and Linux, then plug the USB end into the host PC.

Connect Ethernet. Plug the Ethernet cable into the board's J19 connector, and run the other end to the host PC.



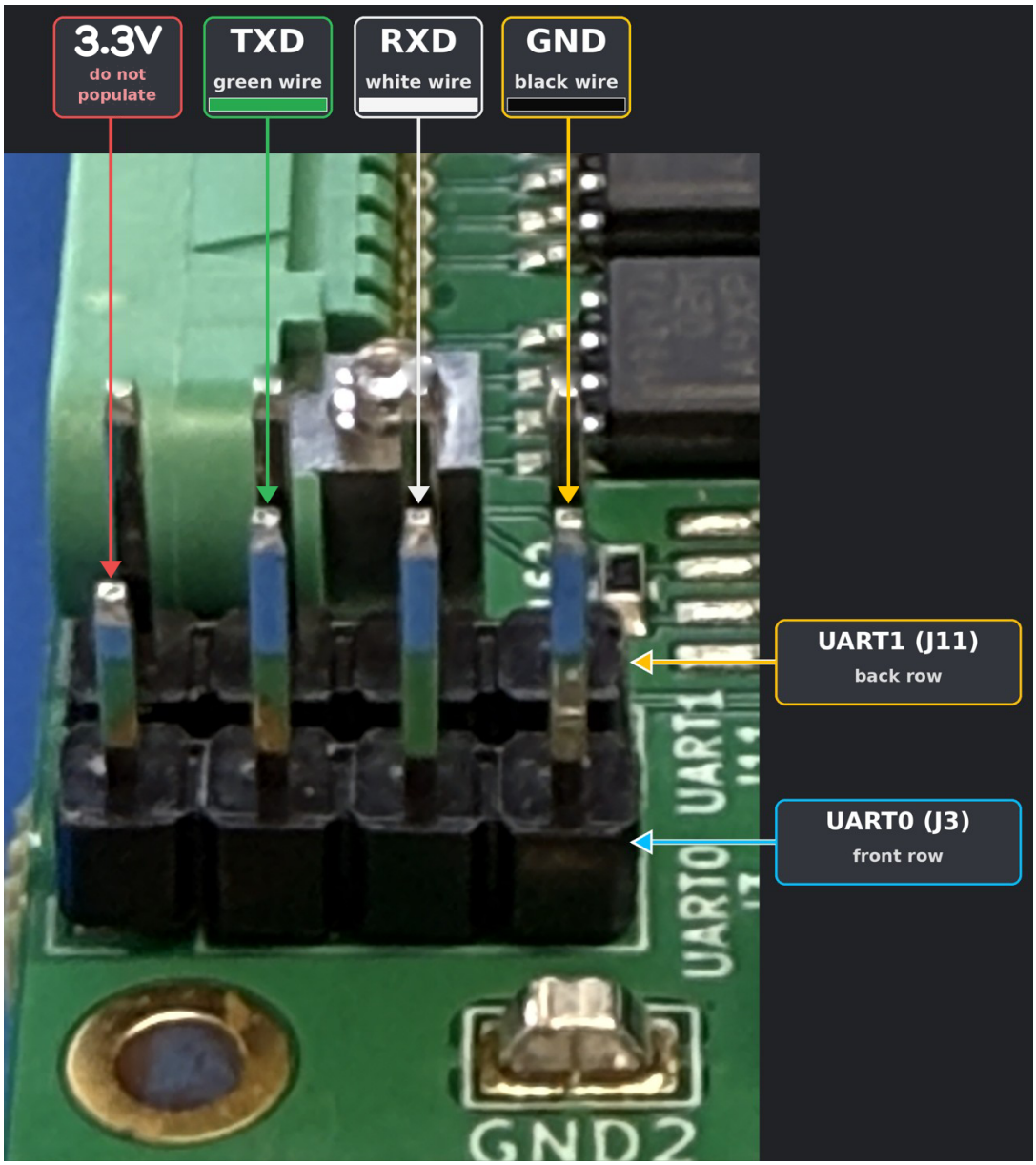
Serial consoles. The UART headers sit together near the board's lower-left corner: UART0 (J3) and UART1 (J11). The photo shows a USB-to-serial connector seated on UART1.

Ethernet. The cable goes into connector J19 at the board edge, next to the two HALO magnetics modules.

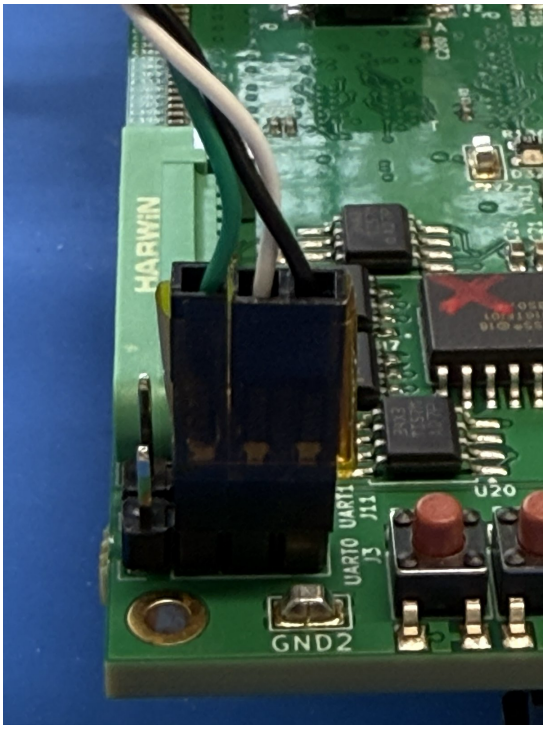
Serial cable wiring. The cable is a 3.3 V TTL USB-to-UART cable (PL2303TA, USB A male, 1x4 female 2.54 mm socket). It carries one UART, so use one cable per UART header. The board header takes only three of the four wires, so leave the red 5 V wire out of the socket.

Header position (left to right, as photographed)	Board signal	Cable wire	Note
1	3.3V	Red (VCC)	Do not populate
2	TXD	Green	Cable RXD
3	RXD	White	Cable TXD
4	GND	Black	Cable GND

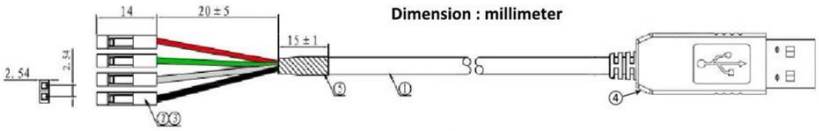
Signals are named from the board's side, so the board's TXD meets the cable's RXD.



The two UART headers, labeled. Both use the same pin order, left to right as photographed with the board edge at the bottom: 3.3V (leave unpopulated), TXD, RXD, GND. The back row is UART1 (J11) and the front row is UART0 (J3).

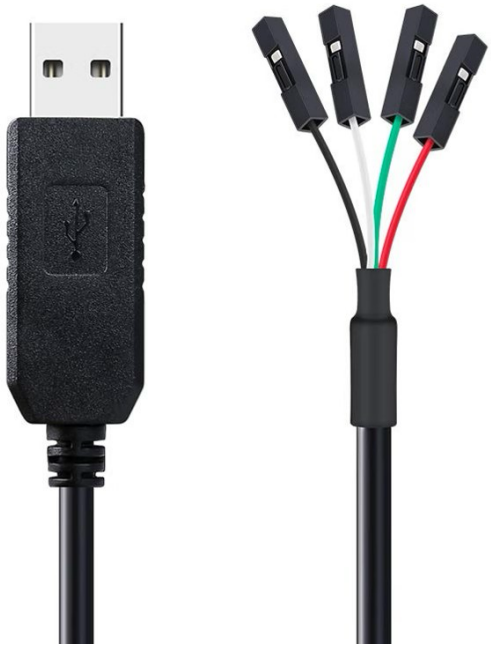


The cable's socket seated on a UART header. The green, white, and black wires sit on pins 2 to 4, and the leftmost pin (3.3V) is left open.



① PVC Jacket
② 2.54mm pitch socket
③ 2.54mm 1*4p terminal ,female
④ USB A male, PL2303TA TTL 3.3V PCBA inside
⑤ Heat shrink tube, black, OD=5.0

1*4P Female Socket	Name	Colour	Description
Pin 1	TXD	White	Transmit Asynchronous Data
Pin 2	RXD	Green	Receive Asynchronous Data
Pin 3	GND	Black	Device ground supply
Pin 4	VCC	Red	+5V



The USB-to-UART cable and its own pin table. Its socket pins are numbered differently from the board header, so use the table above for placement.

Open two Tera Term windows, one per COM port that the USB connection creates. It is also acceptable to use one Tera Term window and just swap between the 2 UARTs. Tera Term will automatically pick up the 2nd UART comms without any intervention.

Use these settings on both:

- Baud: 115200
- Data: 8 bit
- Parity: none
- Stop: 1 bit
- Flow control: none

Window	COM port	Shows
Tera Term Window 1 (UART0)	Port showing the HSS boot log	UART0: HSS bootloader, hardware initialization and boot logs
Tera Term Window 2 (UART1)	The other port	UART1: OpenSBI banner, U-Boot prompt, then the Linux console

Which port is which. Windows assigns the COM numbers itself, and they differ between PCs and USB hubs, so identify the ports by what they print once the board is programmed (Step 4). Open both, then reset the board. The window that prints the HSS boot messages is UART0 (Window 1). The other is UART1 (Window 2), which prints DDR training and the OpenSBI banner, then the U-Boot prompt.

Step 4: Program the FPGA design and firmware

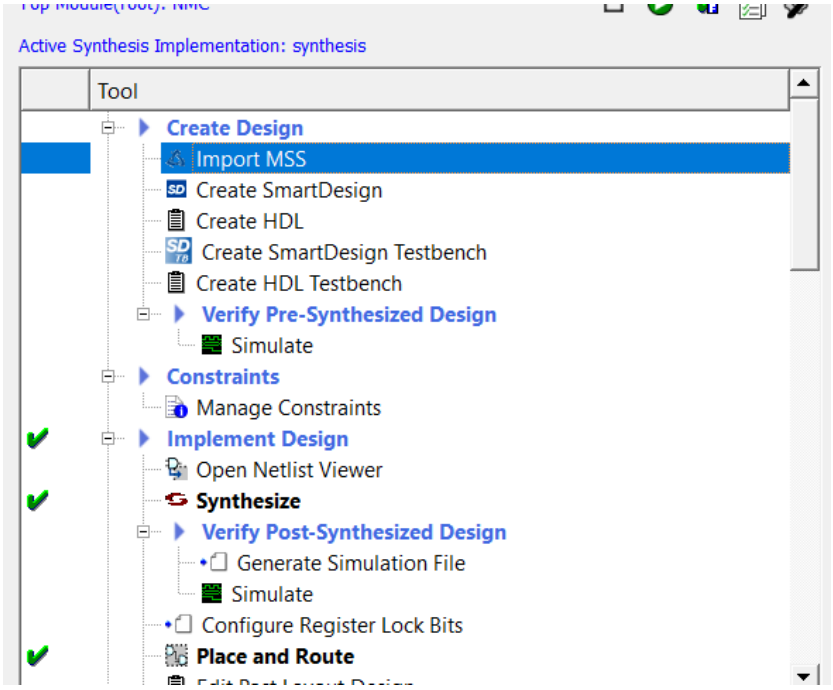
Edit the MSS configuration, build the Libero design, write application code in SoftConsole, and program the result to the board.

A. Configure the MSS

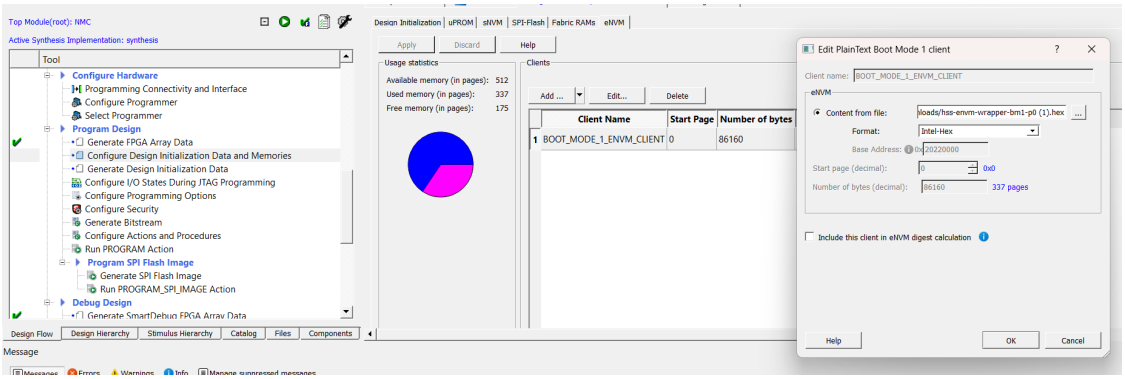
1. Open the `.cfg` file from the `PF_MSS_CFG` folder in the MSS Configurator and edit the RISC-V MSS settings.
2. Press **Generate** to write the `.cxz` and `.xml` output files.

B. Build the design in Libero SoC

1. In the design flow, run **Create Design → Import MSS** and select the new `.cxz`.
2. Work down the flow as shown in the guide's screenshots: **Create SmartDesign**, **Manage Constraints**, **Synthesize**, **Place and Route**.
3. Under **Program Design → Configure Design Initialization Data and Memories → eNVM**, add or edit a client (the guide's example is `BOOT_MODE_1_ENVM_CLIENT`). Choose **Content from file**, format **Intel-Hex**, and select your `.hex`.



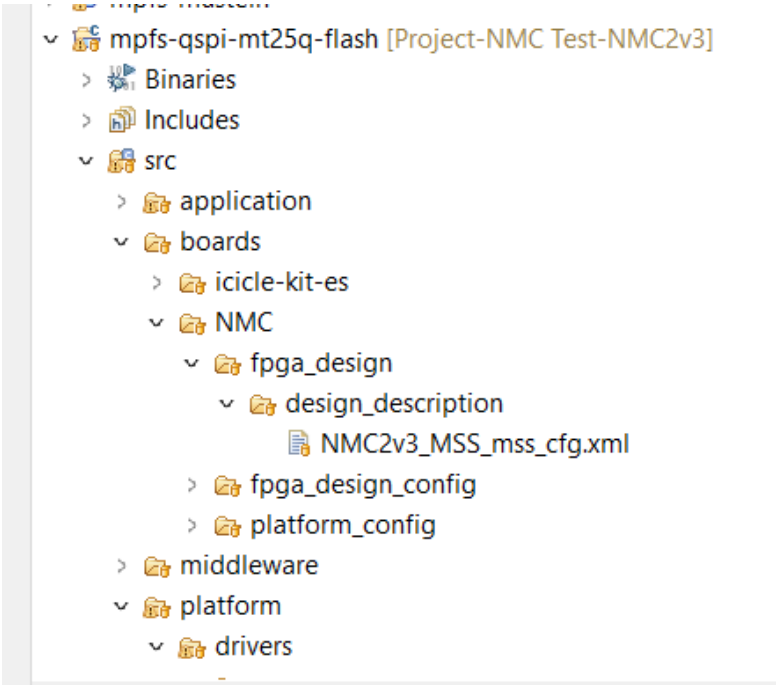
Libero design flow: Create Design → Import MSS is where the `.cxz` goes.



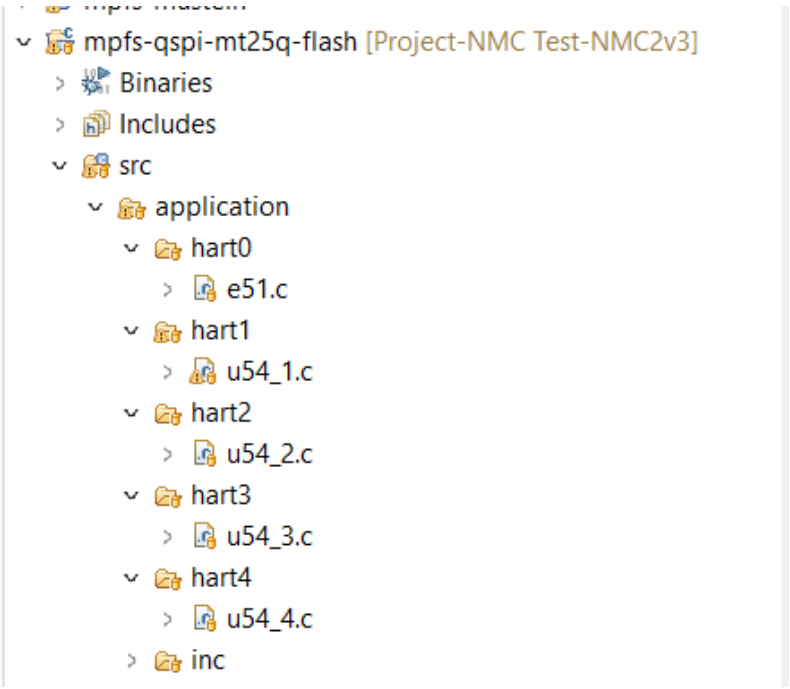
The `.hex` import location: Program Design → Configure Design Initialization Data and Memories → eNVM, then Add a client and select the Intel-Hex file.

C. Write application code in SoftConsole

1. Open the SoftConsole project from the `SoftConsole` folder. The guide's example is `mpfs-qspi-mt25q-flash`.
2. Drag the new `.xml` into the project's `fpga_design/design_description` folder and delete the default `.xml`, so only one remains.
3. Put application code under `src/application`. Hart0 (`e51.c`) mainly handles system startup, so use hart1 to hart4 (`u54_1.c` to `u54_4.c`) for your code.
4. Build to produce the `.hex`, then import it into Libero as in B3.



SoftConsole: the MSS `.xml` sits under `src/boards/NMC/fpga_design/design_description`. Replace the default one with yours.



Application code lives in `src/application`. Hart0 (`e51.c`) handles startup, so use hart1 to hart4.

D. Program the board

1. Connect the FlashPro to the host PC (USB) and to the NMC's ribbon-cable connector, as pictured in Step 3.
2. In Libero, under **Program Design**, run **Generate FPGA Array Data** and **Generate Bitstream**.
3. Run **Run PROGRAM Action** to write the design and eNVM contents to the board.

Step 5: Build or obtain the Linux images

You need `payload.bin` (HSS-wrapped U-Boot) and `sdcard.img`; either build them with Buildroot (about 1 hour the first time) or skip to Step 6 with the prebuilt files. `sample/payload.bin` in the BSP repo is ready to flash, and a prebuilt `sdcard.img` is on the BSP repo's Releases page.

Build everything inside the WSL2 Linux filesystem (for example `~/NMC-Linux`), not under `/mnt/c/...`, which is much slower and can cause path problems.

A. Install host dependencies (WSL2 / Ubuntu 22.04 LTS)

```
sudo apt-get install subversion build-essential bison flex gettext \
  libncurses5-dev texinfo autoconf automake libtool mercurial git-core \
  gperf gawk expat curl cvs libexpat-dev bzr unzip bc python3-dev \
  wget cpio rsync xxd bmap-tools libgnutls28-dev libgl1
```

B. Set up the workspace

```
mkdir -p ~/NMC-Linux && cd ~/NMC-Linux
git clone https://github.com/linux4microchip/buildroot-external-microchip.git
git clone https://git.busybox.net/buildroot -b 2025.02.11
cp -r /mnt/c/Users/<you>/Downloads/mpfs-nmc-linux-bsp ~/NMC-Linux/
```

Adjust the last path to wherever you extracted the BSP zip. Make sure no shell environment variable contains stray spaces, tabs, or newlines; Buildroot is picky about this.

C. Build the baseline Icicle Kit image. The NMC uses the same MPFS250T chip as the Icicle Kit, so `icicle_defconfig` is the right starting point.

```
cd ~/NMC-Linux/buildroot
BR2_EXTERNAL=../buildroot-external-microchip/ make icicle_defconfig
make
```

D. Apply the NMC overlay. Run this only after C finishes, because `output/build/uboot-custom/` and `output/build/linux-custom/` do not exist before then. Stay in `~/NMC-Linux/buildroot`.

```
# Apply configs
cp ~/NMC-Linux/mpfs-nmc-linux-bsp/buildroot/.config .config
cp ~/NMC-Linux/mpfs-nmc-linux-bsp/uboot/.config output/build/uboot-custom/.config
cp ~/NMC-Linux/mpfs-nmc-linux-bsp/linux/.config output/build/linux-custom/.config

# Overwrite DTS files in the build trees
cp ~/NMC-Linux/mpfs-nmc-linux-bsp/uboot/mpfs-nmc-uboot.dts \
   output/build/uboot-custom/dts/upstream/src/riscv/microchip/mpfs-icicle-kit.dts
cp ~/NMC-Linux/mpfs-nmc-linux-bsp/linux/mpfs-nmc-linux.dts \
   output/build/linux-custom/arch/riscv/boot/dts/microchip/mpfs-icicle-kit.dts
cp ~/NMC-Linux/mpfs-nmc-linux-bsp/linux/mpfs-nmc-linux.dts \
   output/build/linux-custom/arch/riscv/boot/dts/microchip/mpfs-icicle-kit-prod.dts

# Update the kernel build timestamp and clear cached artifacts
touch output/build/linux-custom/init/main.c
rm -f output/images/payload.bin output/images/mpfs_icicle.itb \
      output/images/boot.vfat output/images/sdcard.img

# Rebuild
make uboot-rebuild
make linux-rebuild
make host-hss-payload-generator-rebuild
make
```

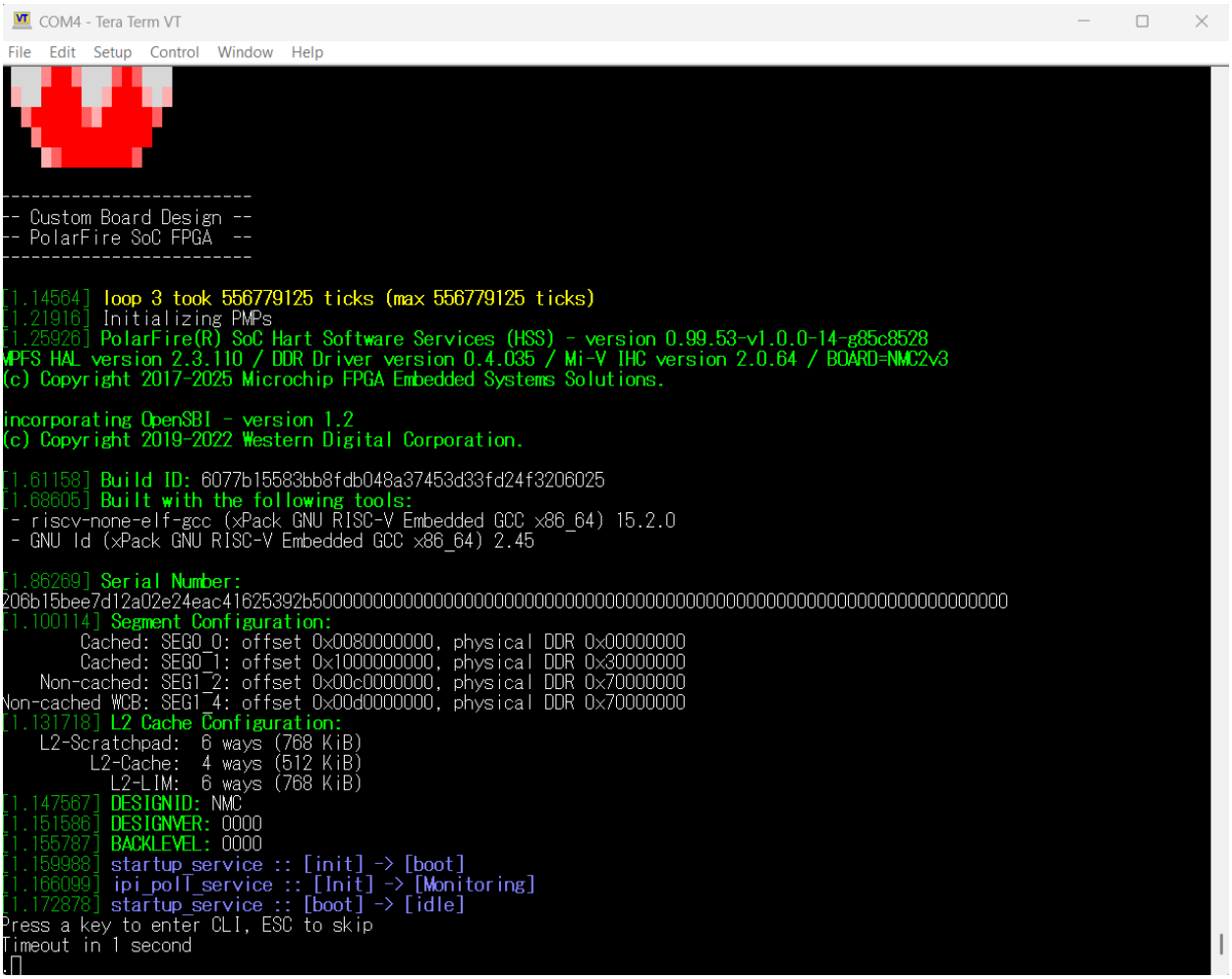
Outputs land in `buildroot/output/images/`: `payload.bin`, `sdcard.img`, `mpfs_icicle.itb`, and `boot.vfat`. To change peripherals instead of applying the saved snapshot, use `make menuconfig`, `make uboot-menuconfig`, or `make linux-menuconfig`.

Step 6: Flash payload.bin to eMMC (HSS, YMODEM)

Interrupt the HSS boot in Tera Term on Window 1 (UART0), send `payload.bin` with YMODEM, and write it to eMMC. Use `buildroot/output/images/payload.bin` or `sample/payload.bin`.

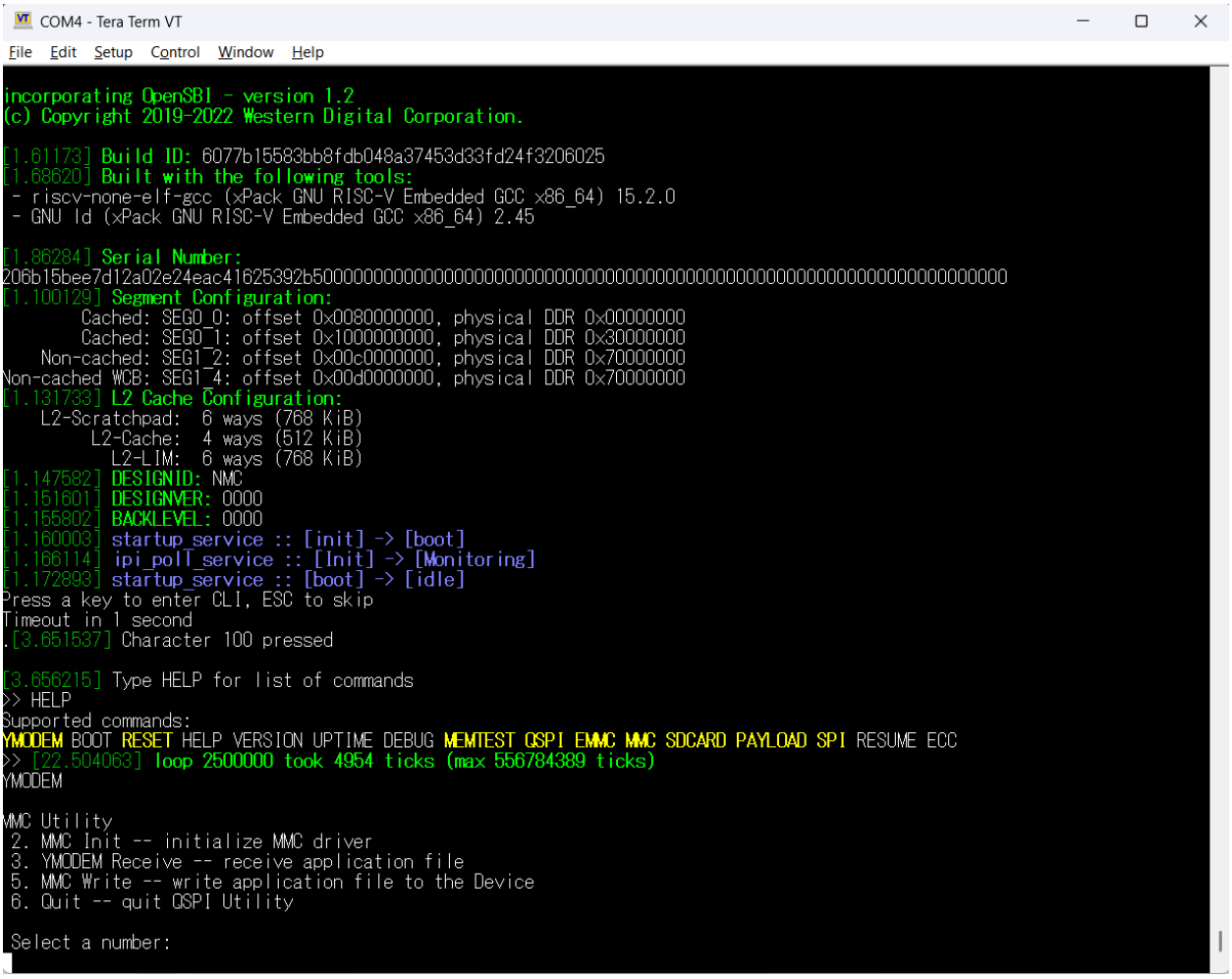
Get to the HSS prompt

1. Power cycle the board or press its Reset button.
2. Immediately press any key in Tera Term Window 1 (UART0) to stop auto-boot and reach the >> prompt.

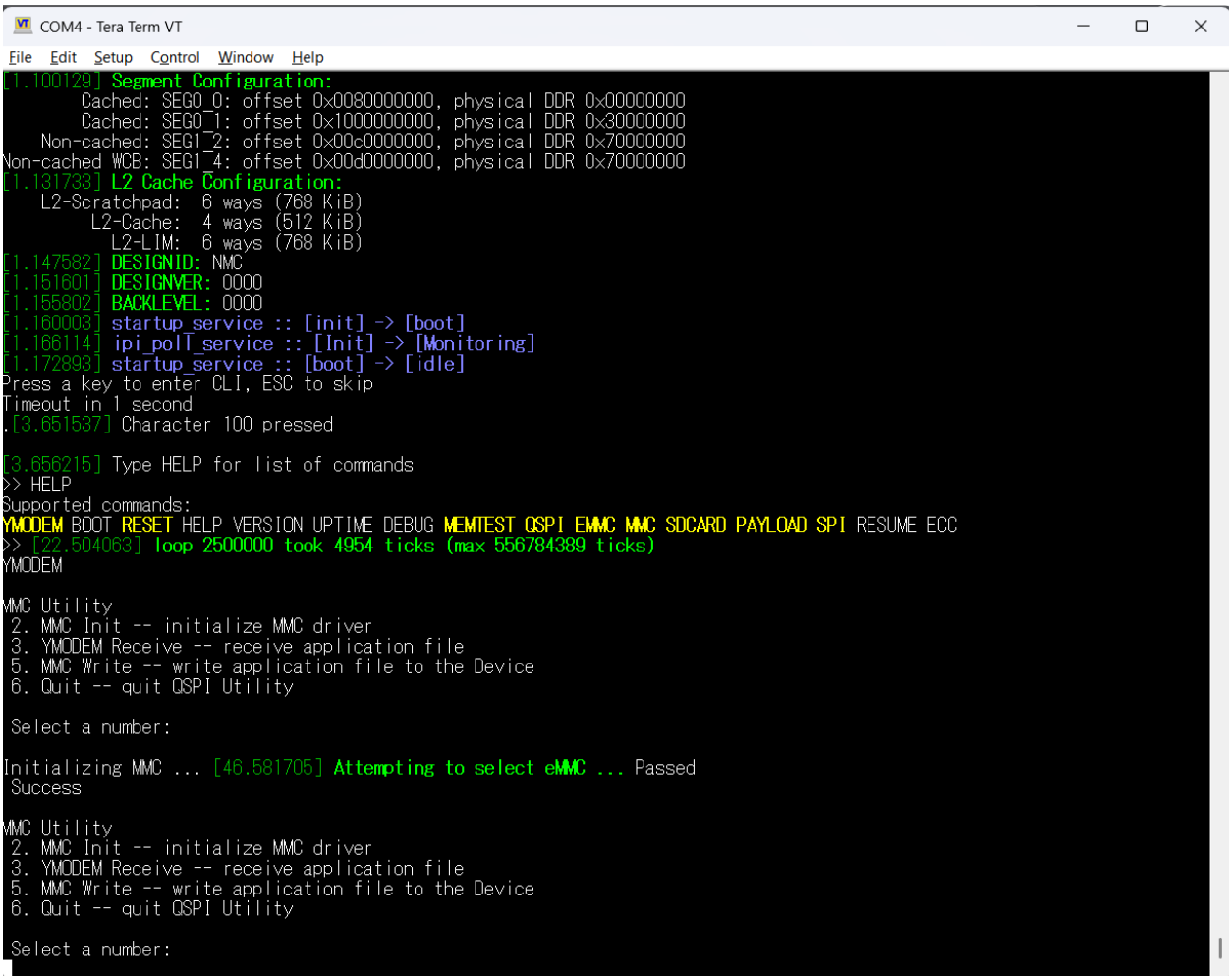


Receive the image

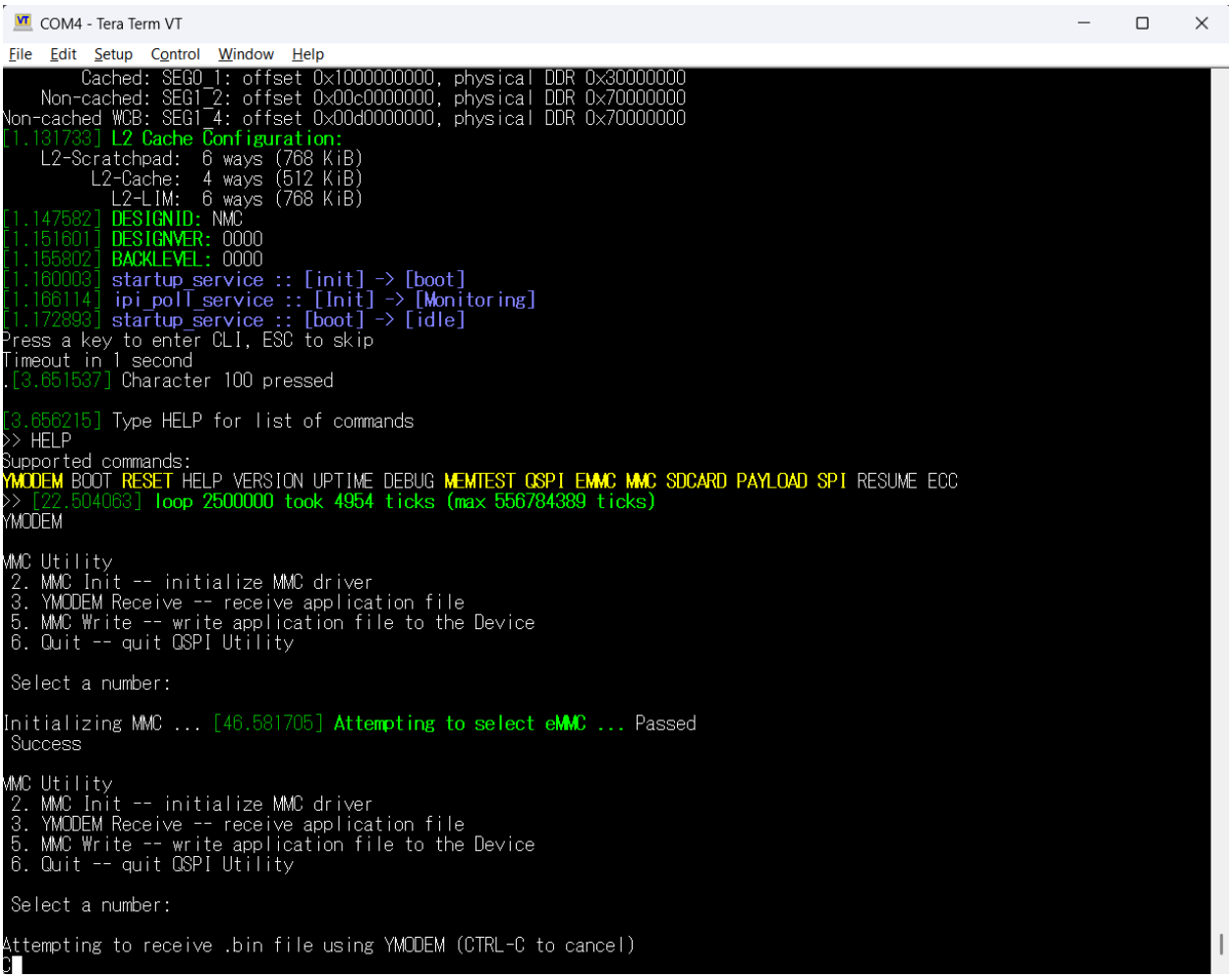
1. Type YMODEM and press Enter.



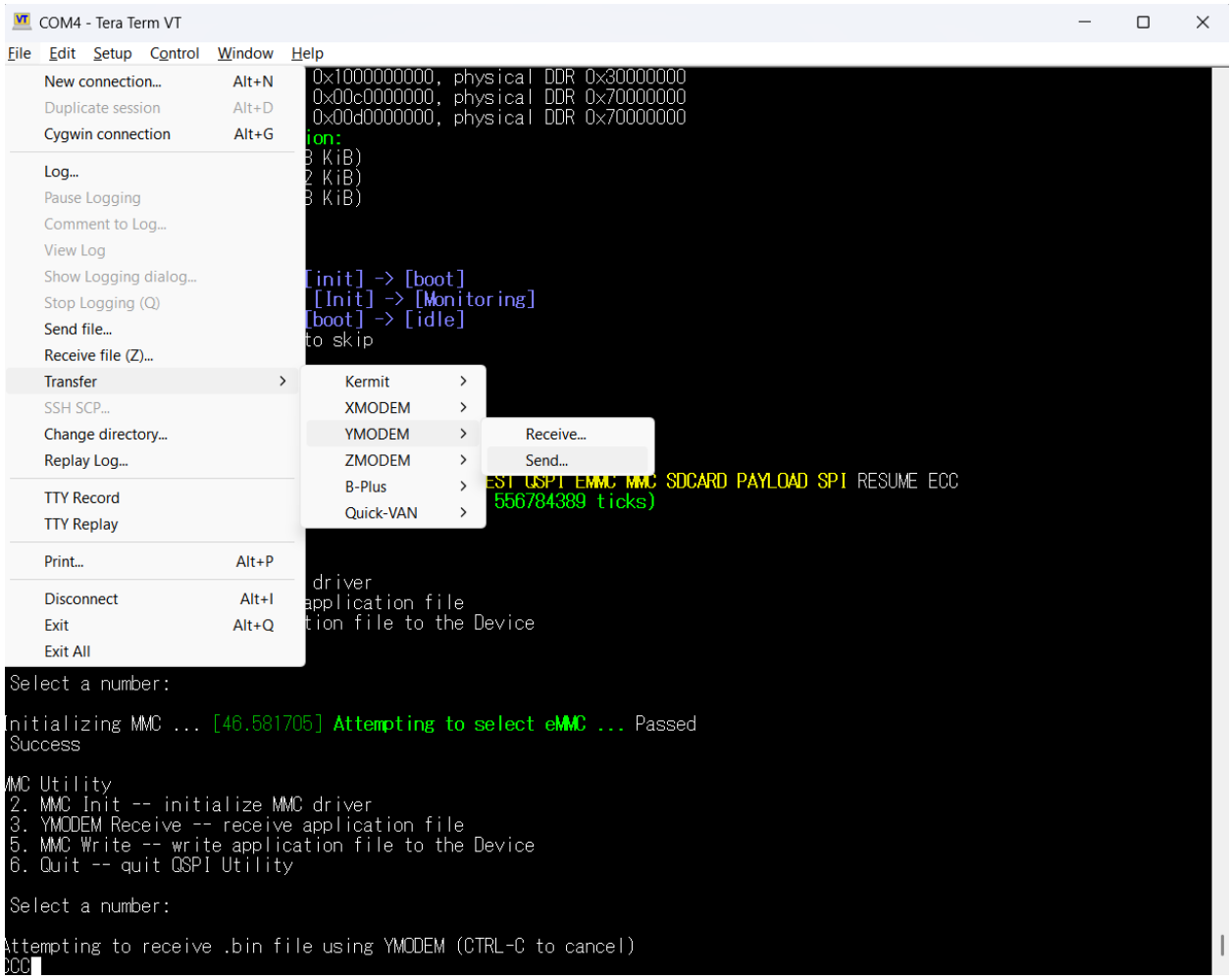
2. Type 2 and press Enter to run **MMC Init**.

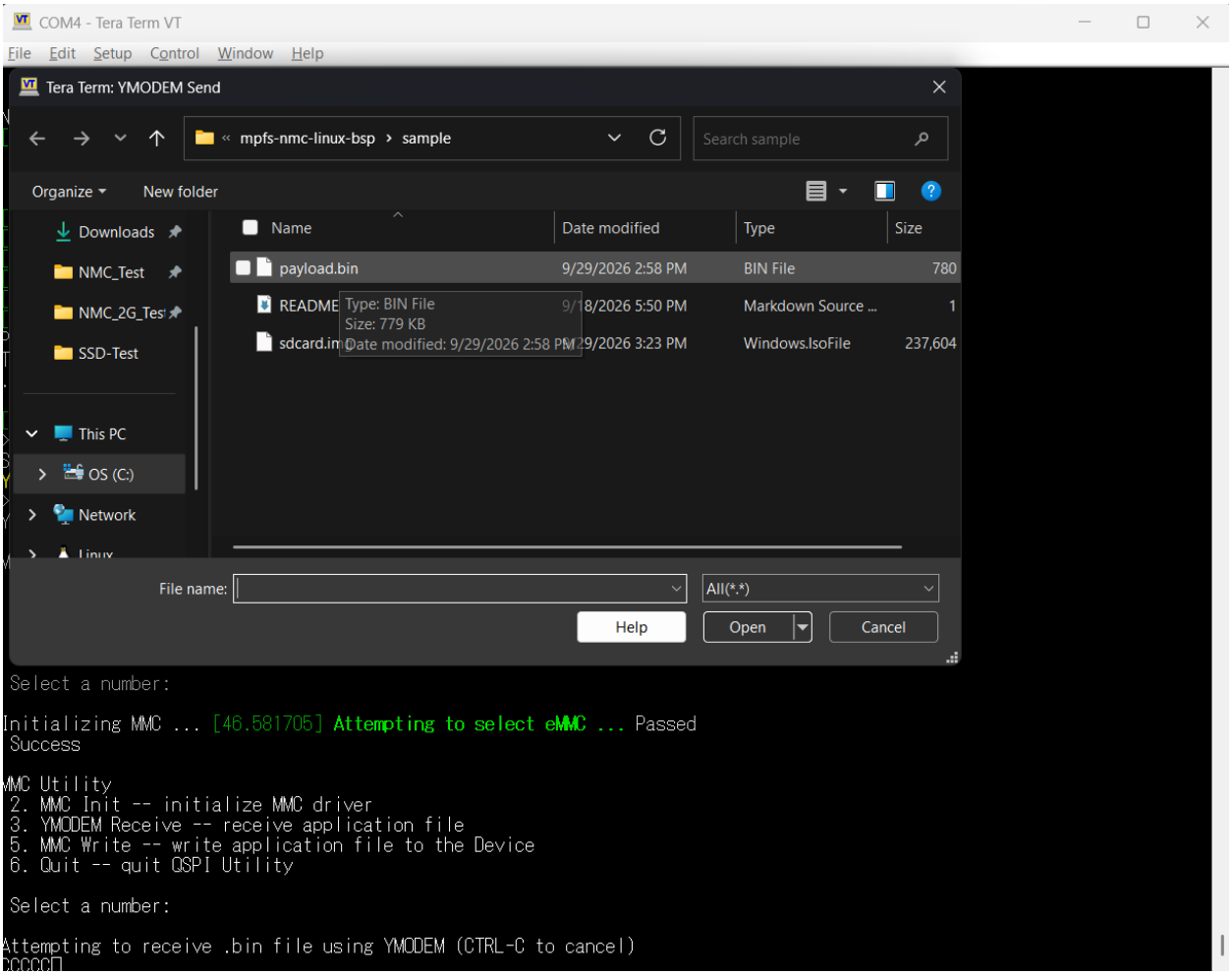


3. Type 3 and press Enter to select **YMODEM Receive**.

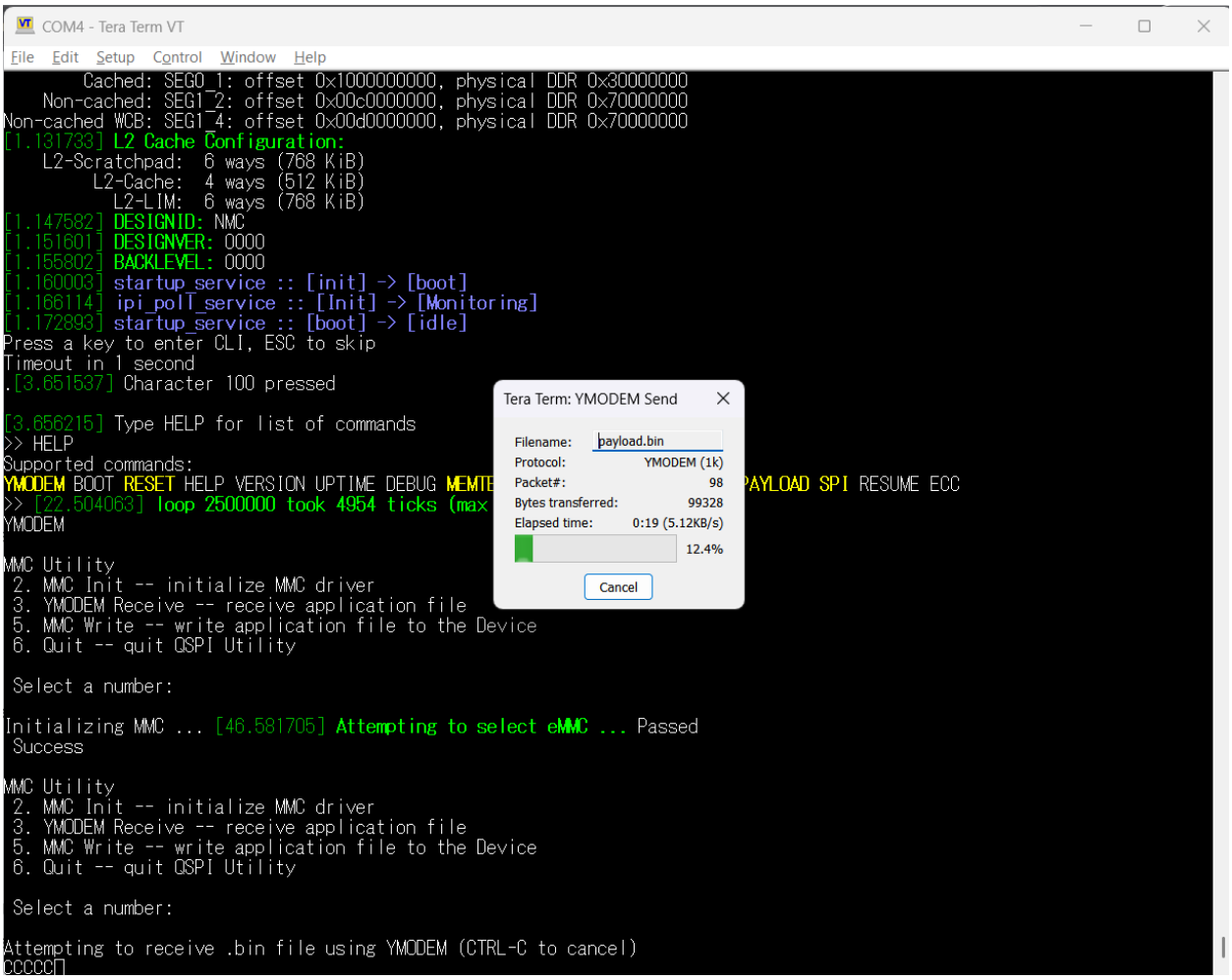


4. In Window 1 (UART0), choose **File → Transfer → YMODEM → Send...**, select `payload.bin`, and click **Open**.




COM4 - Tera Term VT
File Edit Setup Control Window Help
Tera Term: YMODEM Send
mpfs-nmc-linux-bsp › sample
Search sample
Organize
New folder
Downloads
NMC_Test
NMC_2G_Test
SSD-Test
This PC
OS (C:)
Network
Name
Date modified
Type
Size
payload.bin
9/29/2026 2:58 PM
BIN File
780
README
Type: BIN File
Size: 779 KB
Date modified: 9/29/2026 2:58 PM
Markdown Source ...
1
sdcard.im
Windows.IsoFile
237,604
File name:
All(*.*)
Help
Open
Cancel
Select a number:
Initializing MMC ... [46.581705] Attempting to select eMMC ... Passed
Success
MMC Utility
2. MMC Init -- initialize MMC driver
3. YMODEM Receive -- receive application file
5. MMC Write -- write application file to the Device
6. Quit -- quit QSPI Utility
Select a number:
Attempting to receive .bin file using YMODEM (CTRL-C to cancel)

5. Wait for the transfer to finish.

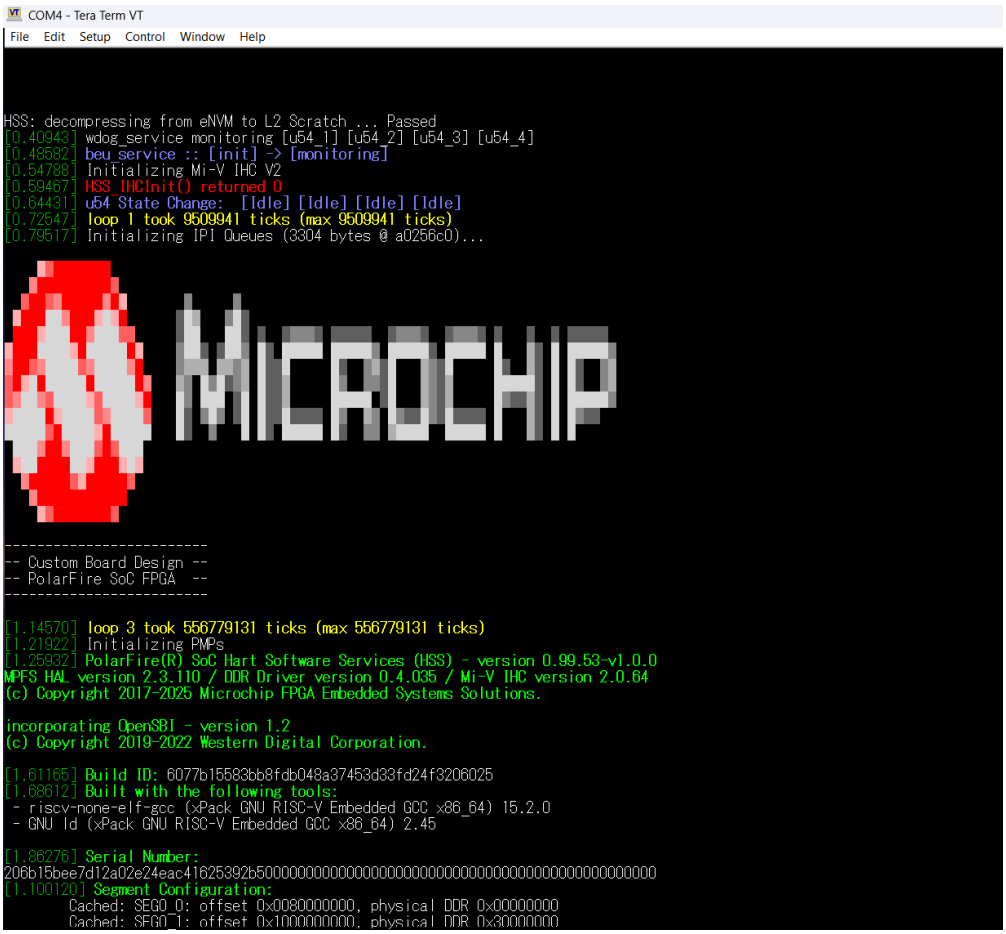


Write it permanently

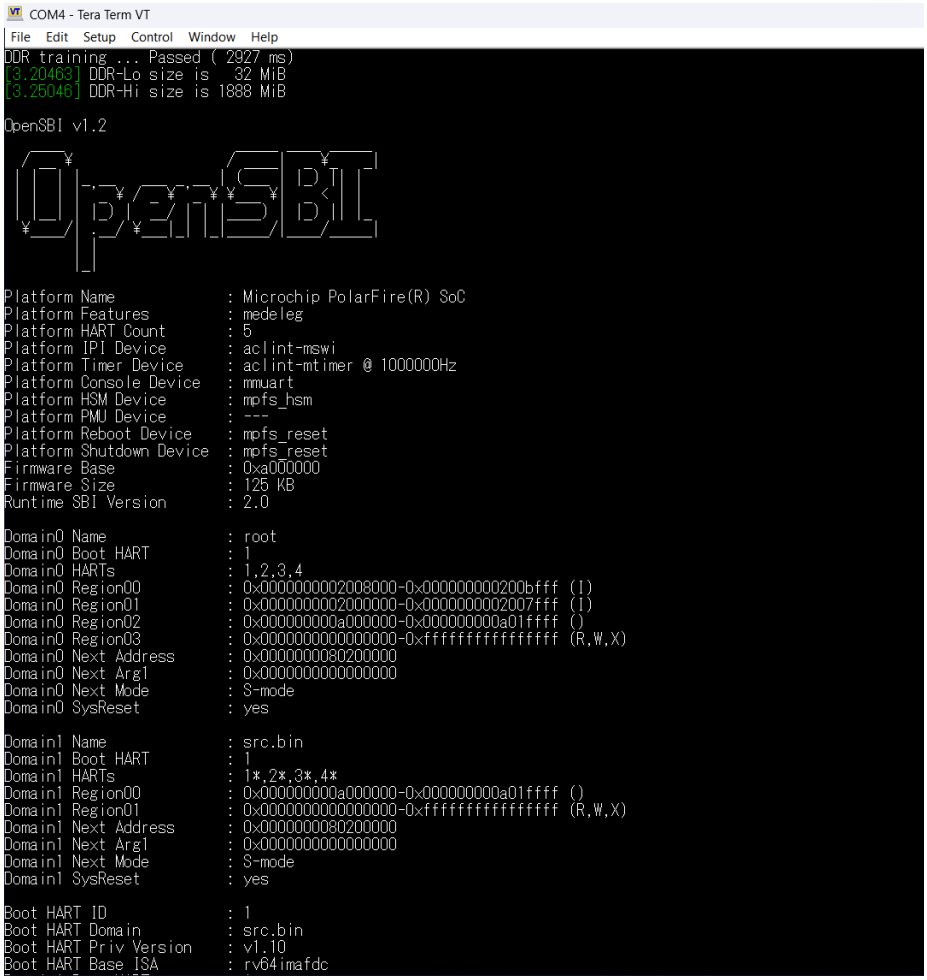
1. Type 5 and press Enter to run **MMC Write**.
2. Type 6 and press Enter to quit the utility.

Checkpoint: after a reset, you should land at a U-Boot prompt in Tera Term Window 2 (UART1). If not, resolve that before Step 7.

Both windows should look like this after `payload.bin` is written and the board resets, before you load the Linux image in Step 7.



UART0 (Window 1): the HSS boot log. It starts with HSS decompressing from eNVM, then shows the Microchip banner and the HSS version and build details.



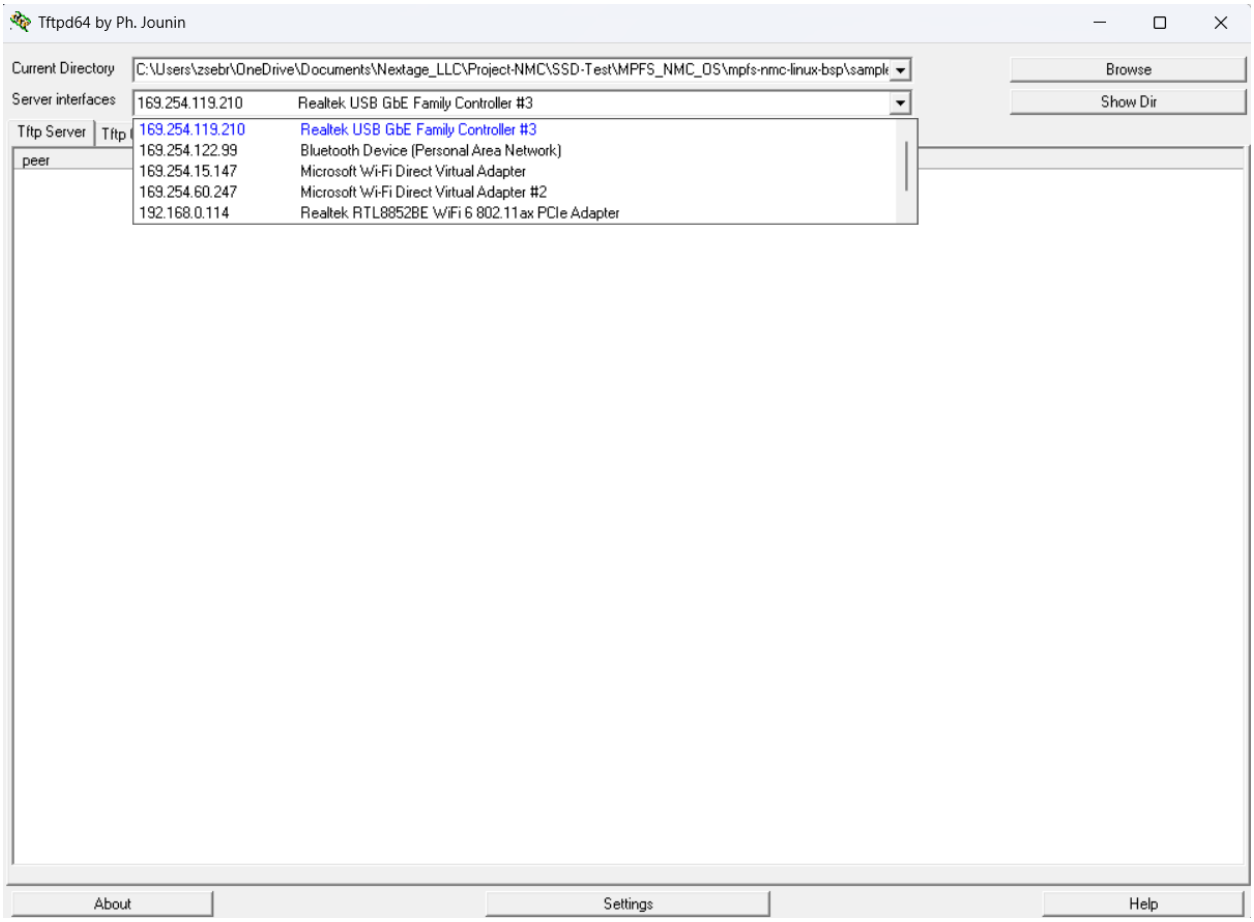
UART1 (Window 2): DDR training, then the OpenSBI banner with platform and domain details, as U-Boot starts.

Step 7: Flash sdcard.img to eMMC (U-Boot, TFTP)

Serve `sdcard.img` from the host over the Ethernet cable, then pull it into RAM from U-Boot and write it to eMMC. Use `buildroot/output/images/sdcard.img` or the file from the Releases page.

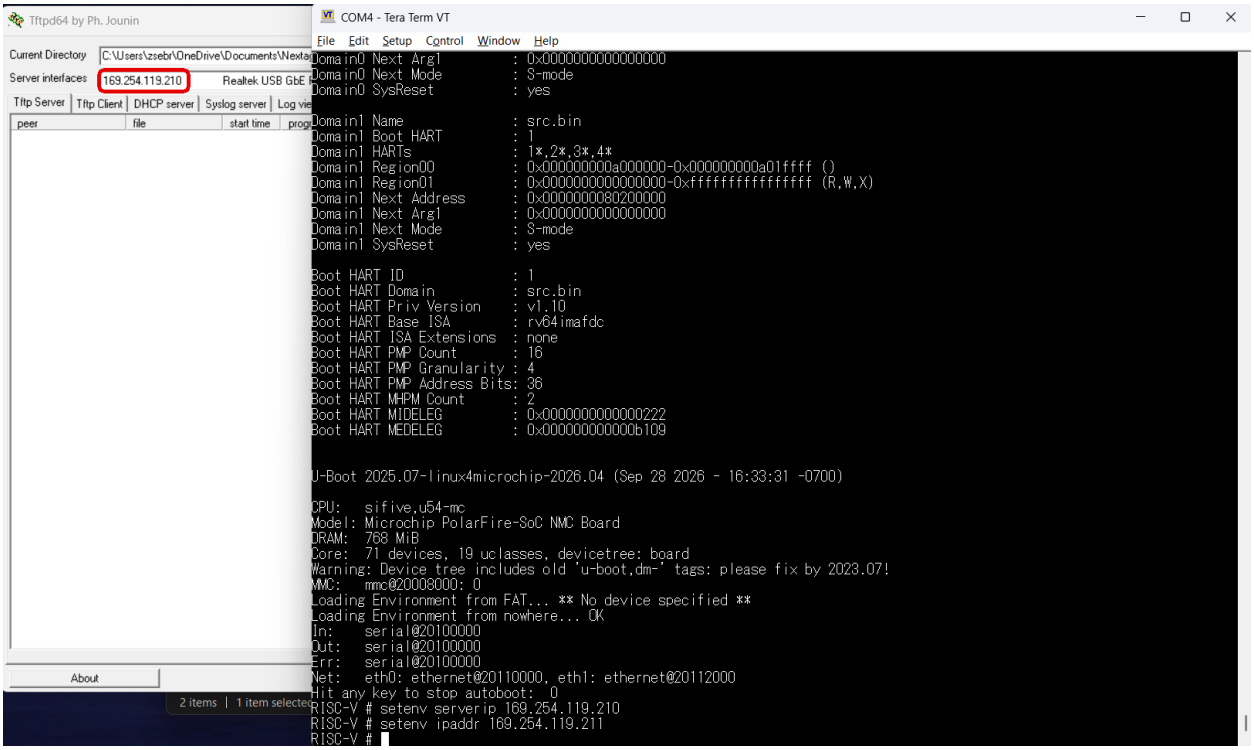
Set up the TFTP server on the host

1. Confirm the Ethernet cable runs from the board's J19 connector to the host PC. Use the eth0 end of the cable.
2. Start a TFTP server such as tftpd64. Point it at the folder containing `sdcard.img` and select the host's Ethernet interface.
3. Use tftp64 to find your Ethernet controller on your machine and write down the IP address. You'll need it for the next steps. In this picture, the IP address of the ethernet port is 169.254.119.210. Yours will be different.



Set the network in Tera Term UART 1 (U-Boot)

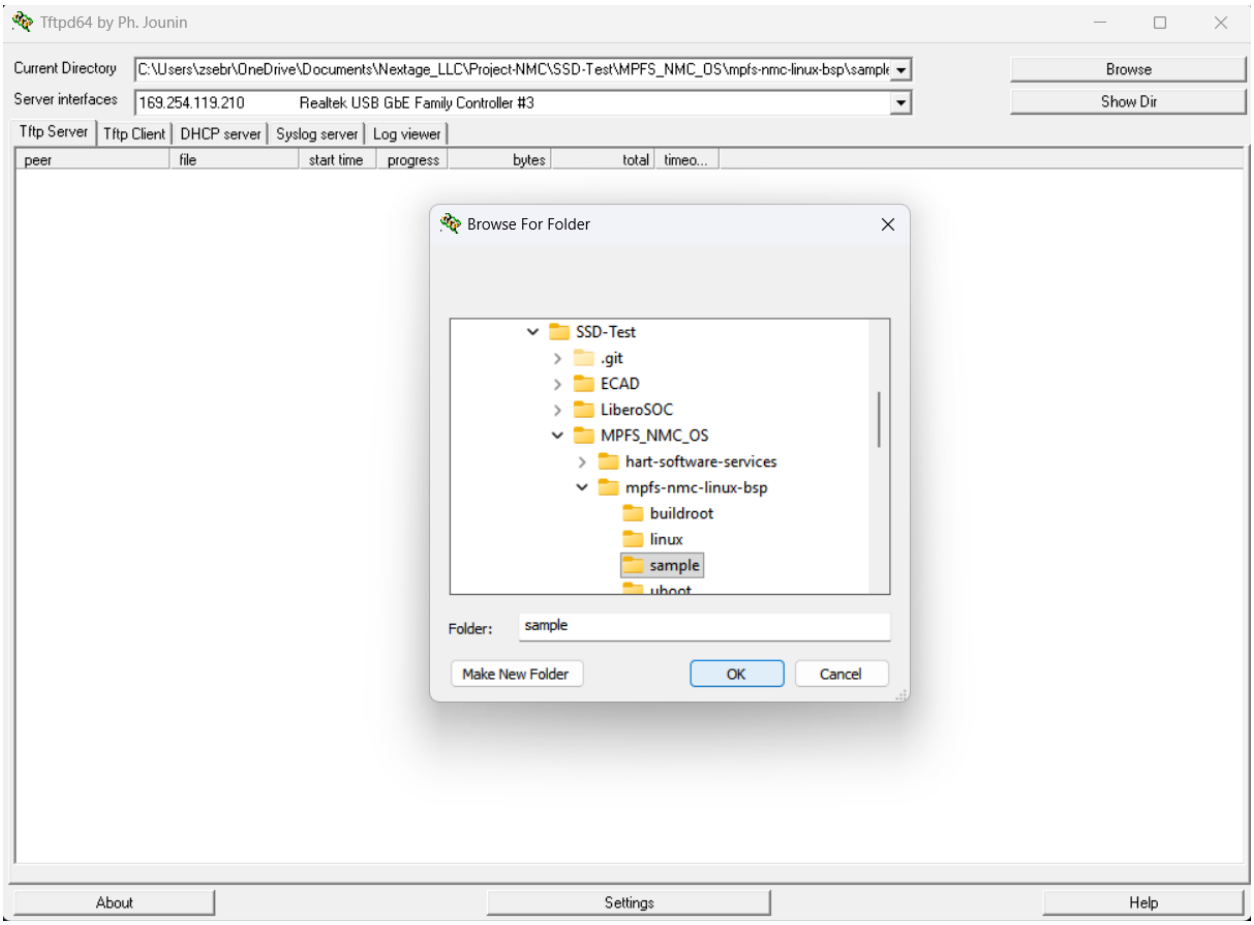
```
setenv serverip 169.254.69.98
setenv ipaddr 169.254.69.99
```

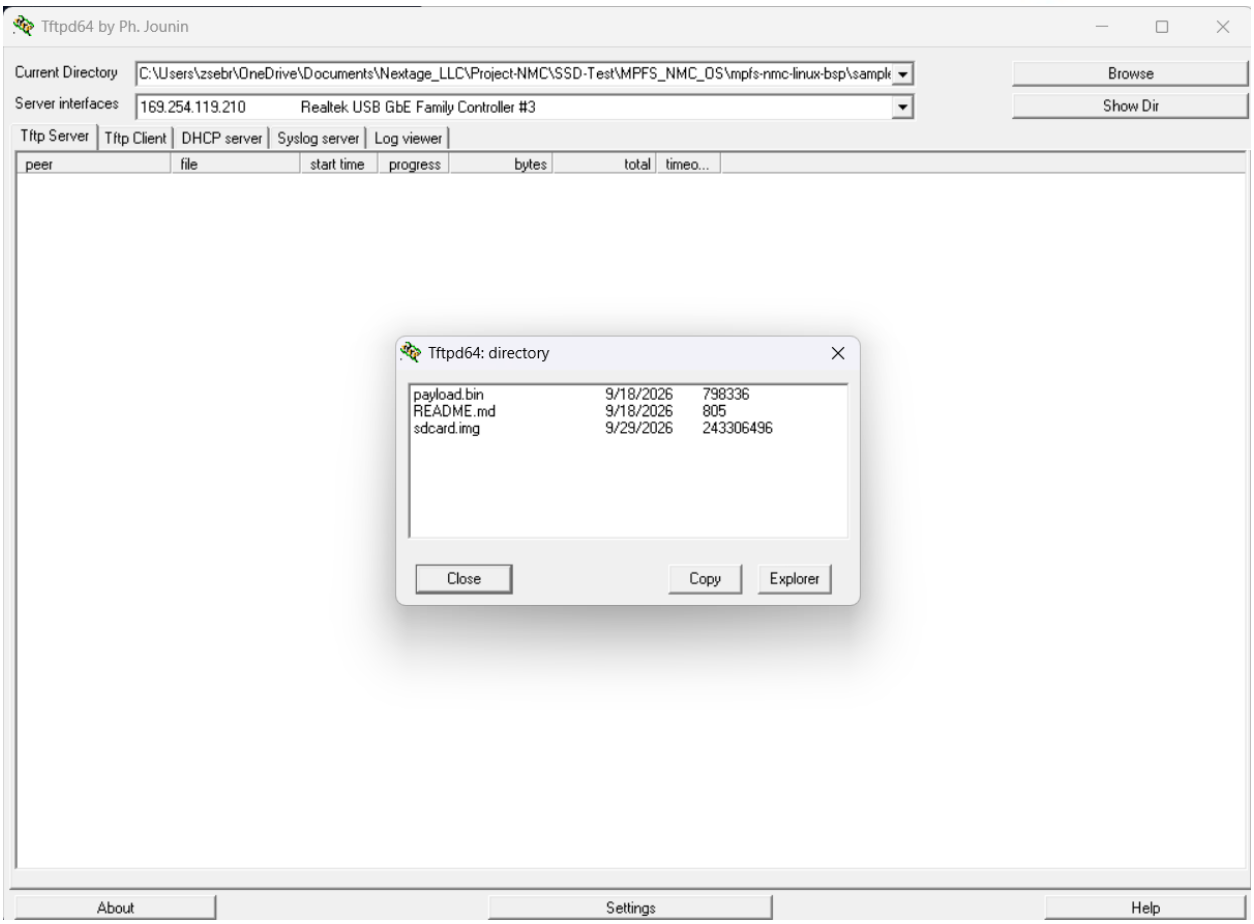


Set `serverip` to your host's Ethernet interface address, and `ipaddr` to an unused address on the same subnet. Set the host's Ethernet adapter to the `serverip` address manually.

Load and write the image

Copy the sdcard image into your sample directory.

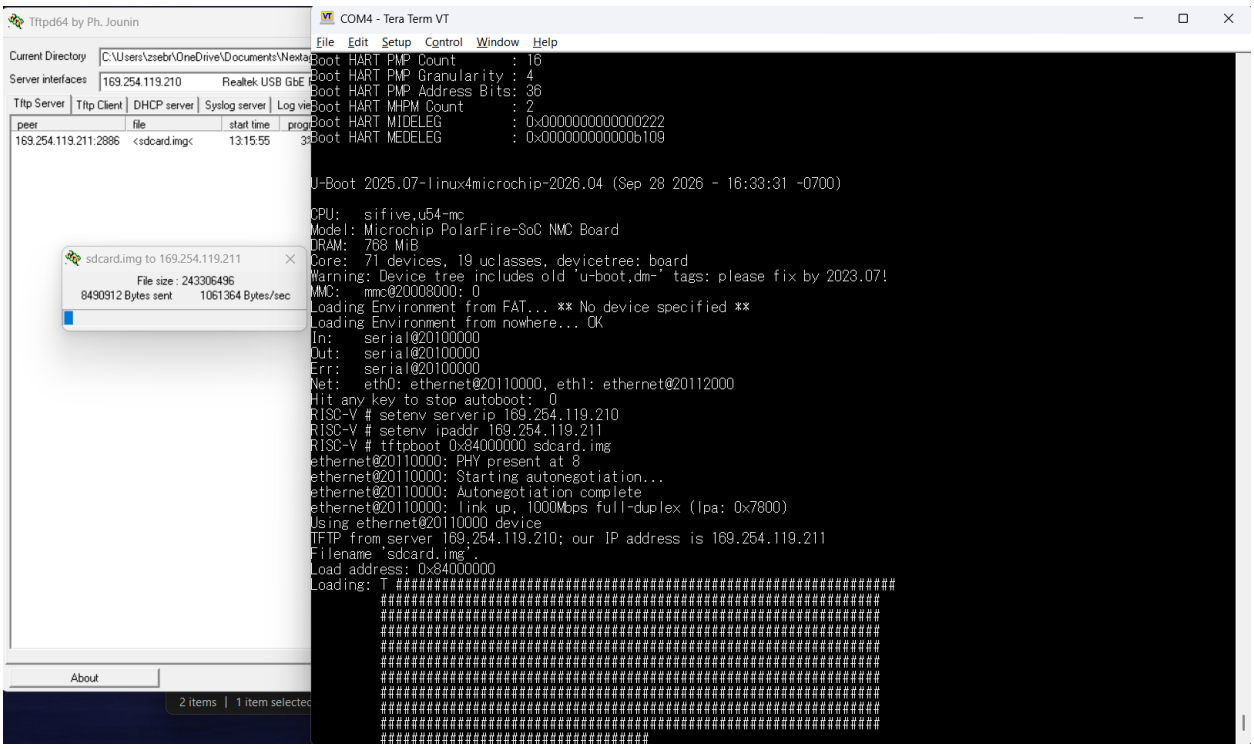


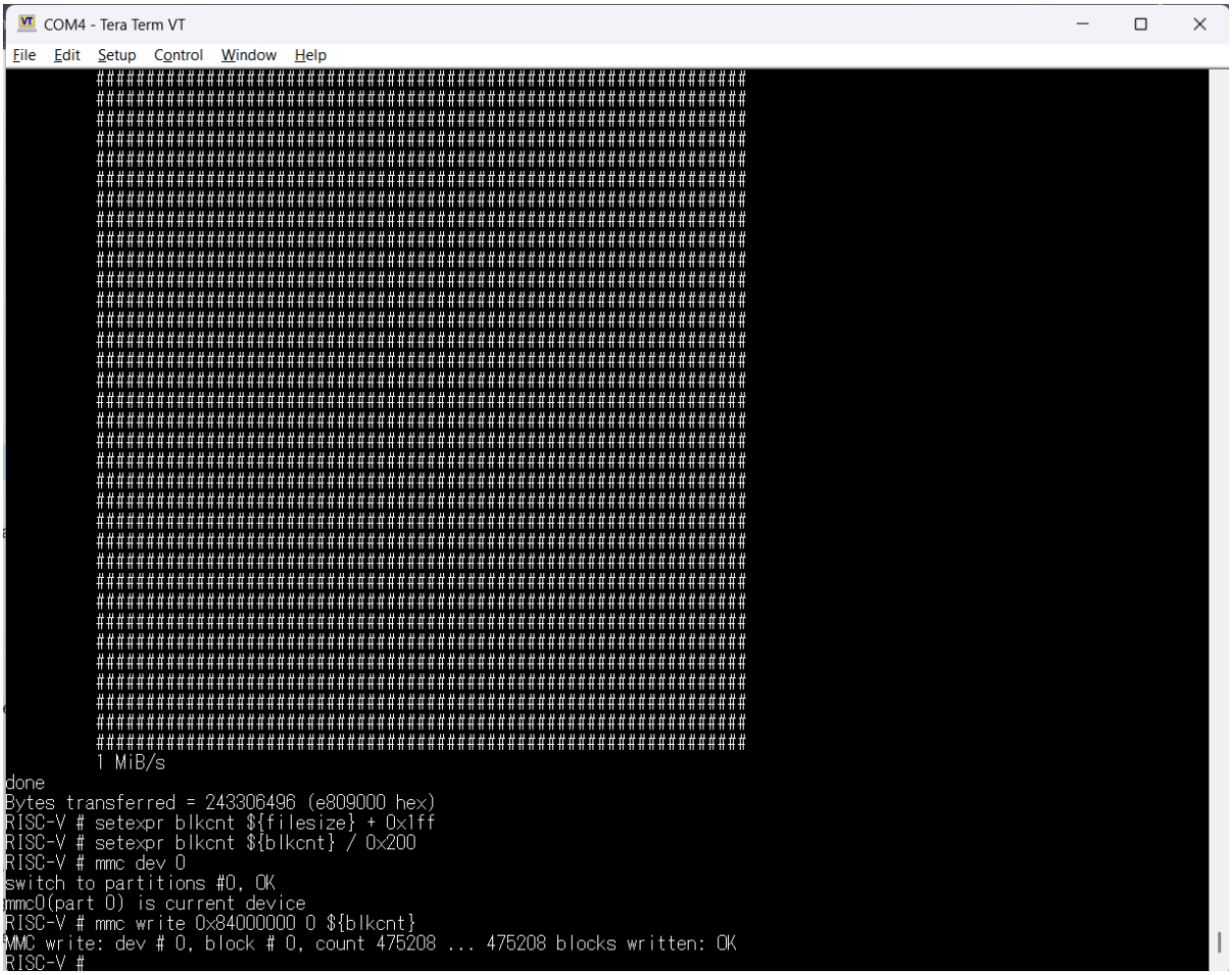

Tftpd64 by Ph. Jounin
Current Directory
C:\Users\zsebr\OneDrive\Documents\Nextage_LLC\Project-NMC\SSD-Test\MPFS_NMC_OS\mpfs-nmc-linux-bsp\sample
Browse
Server interfaces
169.254.119.210 Realtek USB GbE Family Controller #3
Show Dir
Tftp Server
Tftp Client
DHCP server
Syslog server
Log viewer
peer
file
start time
progress
bytes
total
timeo...
Tftpd64: directory
payload.bin 9/18/2026 798336
README.md 9/18/2026 805
sdcard.img 9/29/2026 243306496
Close
Copy
Explorer
About
Settings
Help

```
tftpboot 0x84000000 sdcard.img
setexpr blkcnt ${filesize} + 0x1ff
setexpr blkcnt ${blkcnt} / 0x200
mmc dev 0
mmc write 0x84000000 0 ${blkcnt}
```

(0x84000000 has 6 0s after the 84)

The two `setexpr` lines convert the downloaded file size into a count of 512-byte blocks, rounded up, for `mmc write`.



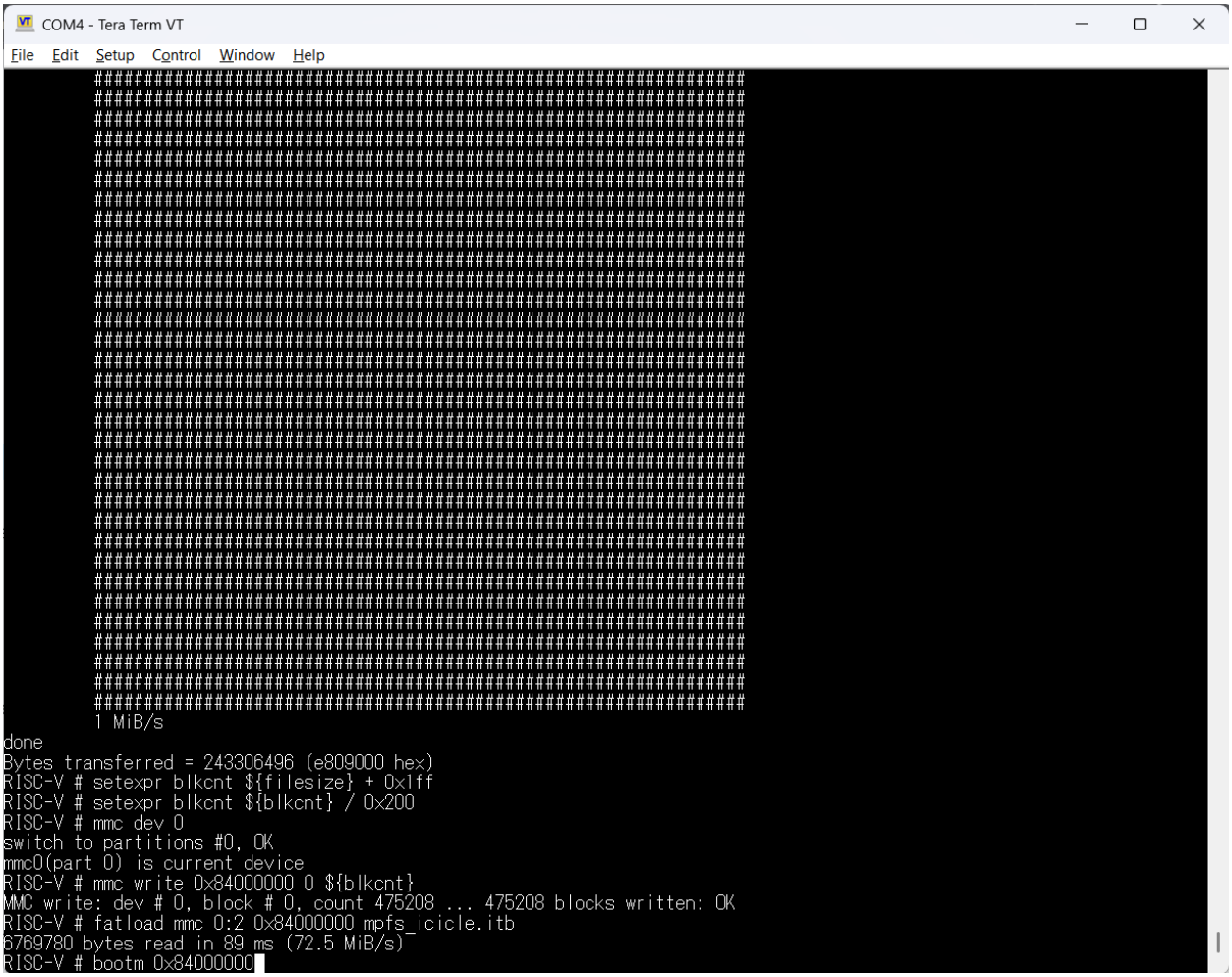


Step 8: Boot Linux and persist auto-boot

Load the kernel image from eMMC in U-Boot (Window 2, UART1), log in, then save `bootcmd` so later resets boot straight into Linux.

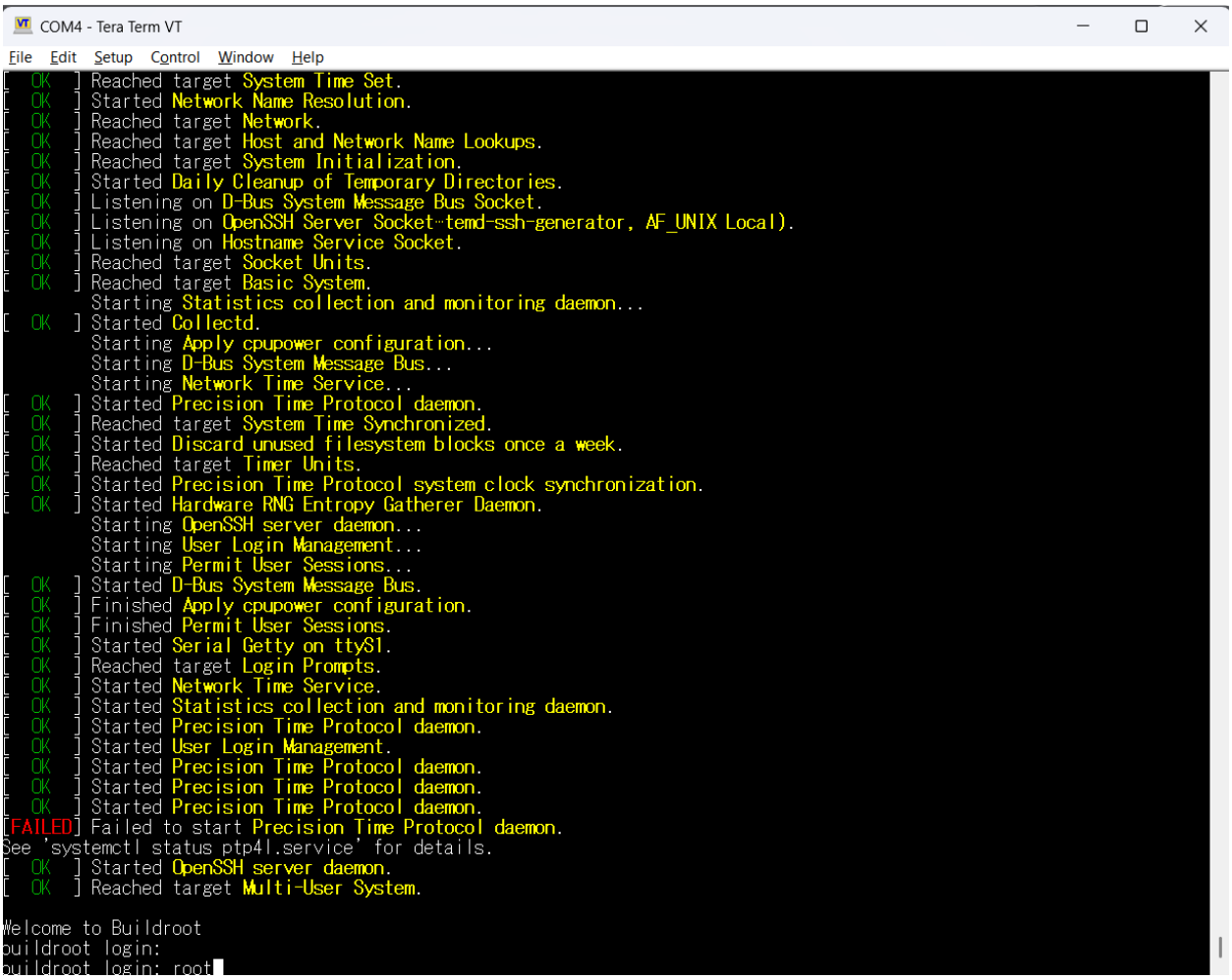
Boot manually

```
fatload mmc 0:2 0x84000000 mpfs_icicle.itb
bootm 0x84000000
```



Once U-Boot is set to load from eMMC by default, typing `boot` or power cycling also works.

Log in. Username `root`, password `root`.



Persist auto-boot. If a reset drops you back at the U-Boot prompt, `bootcmd` has not been saved. Check it:

```
printenv bootcmd
# bootcmd=fatload mmc 0:2 0x84000000 mpfs_icicle.itb; bootm 0x84000000
```

If it matches the line above, run:

```
saveenv
```

Resets will then boot directly into Linux.